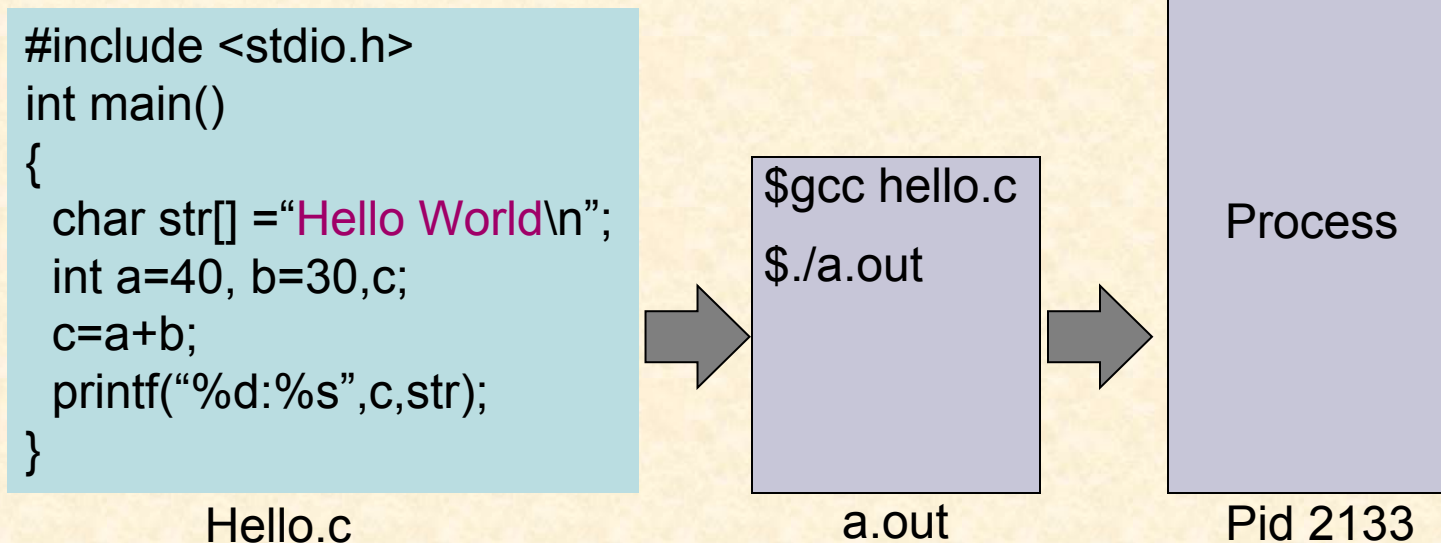


Processes

Sk Subidh Ali
IIT Bhilai

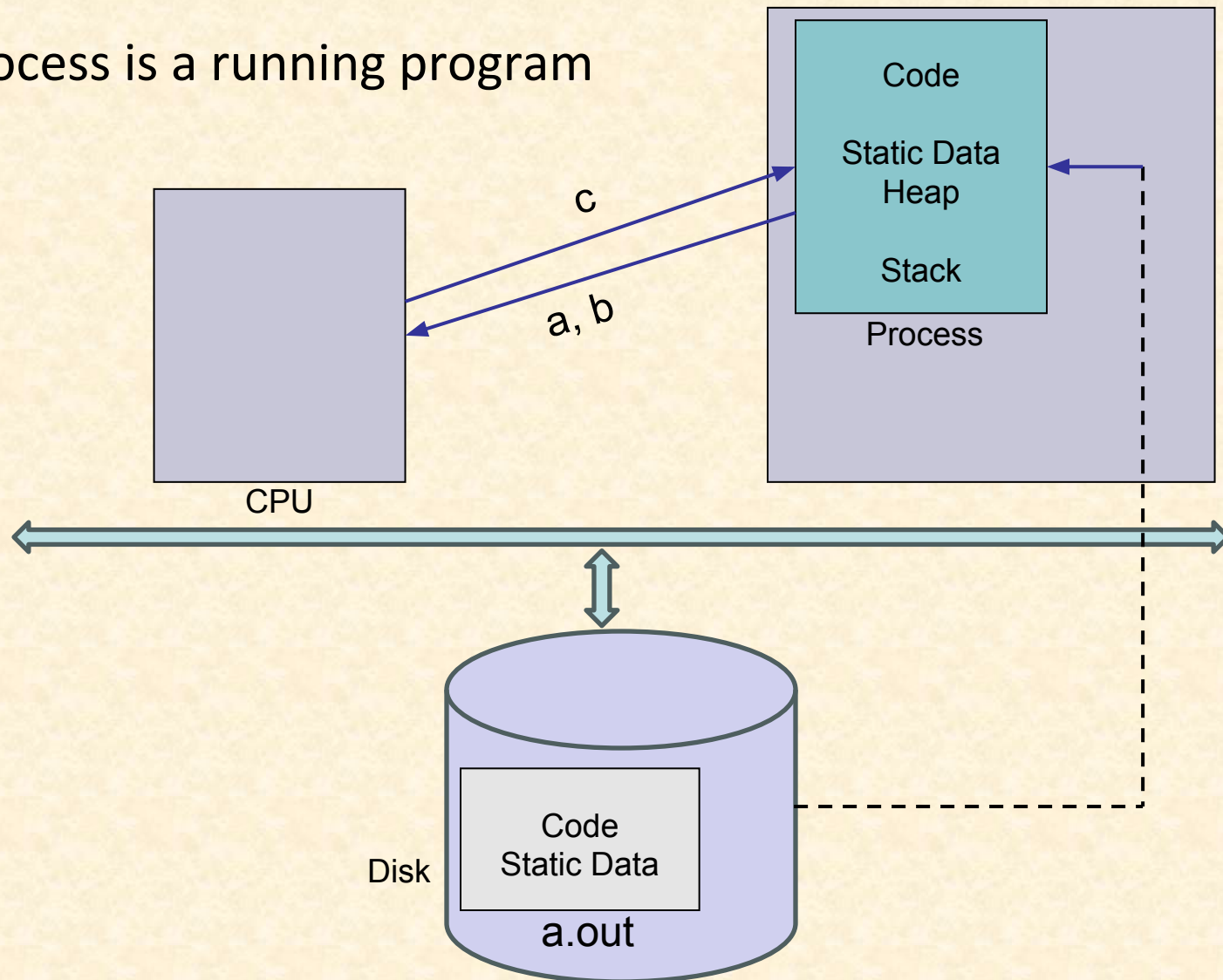
Process

- Process is a running program



Process (hardware view)

- Process is a running program



What info. of a process OS preserves

1. Name: A unique id (process id or pid)
2. Address: memory location (memory image)
 - Code : The binary instruction
 - Data : Keeps global and static variables and constants
 - Stack : For local variable and function calls
 - Heap : For dynamic memory allocation (malloc())
3. State:
 - Registers: PC will show next instⁿ to execute. GPRs will show state of current computation
 - List of open files
 - Related processes: a process may have executed other process to complete certain task
 - I/O communication states

OS Data structures for process

- Process related information stored in PCB (process control block): defined by `struct proc` in xv6
- OS maintains `queues` for the process in:
 - Ready, Blocked, waiting states.

OS Data structures for process

- Process related information stored in PCB (process control block): defined by `struct proc` in xv6

```
struct proc {  
    uint sz;  
    pde_t* pgdir;  
    char *kstack;  
    enum procstate state;  
    int pid;  
    struct proc *parent;  
    struct trapframe *tf;  
    struct context *context;  
    void *chan;  
    int killed;  
    struct file *ofile[NOFILE];  
    struct inode *cwd;  
    char name[16];  
};
```

Size of process memory

Page directory pointer for process

Kernel stack pointer

Ready, running, blocked, etc

Trap frame for current system call

Context of process: ip, bp, sp, fp

Files opened

Current working directory

Executable name

PCB contents varies with OSes

Process States

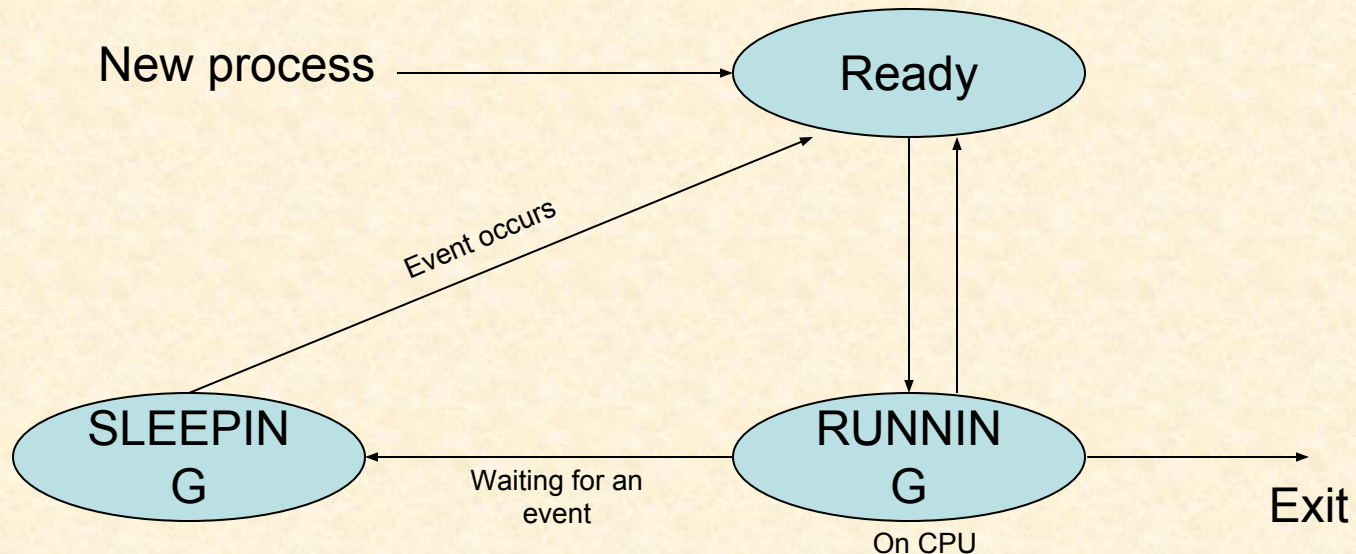
- In general, a process state varies among following four:

Running: The process is currently running in CPU

Ready/Runnable: Ready to run

Blocked/suspended/waiting: Process can not run for the time being

Other states ZOMBIE (later)



State transition example

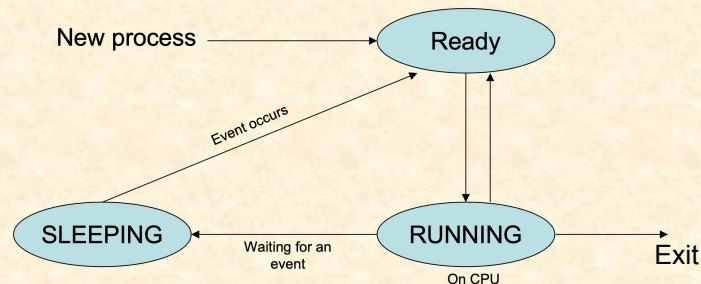
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

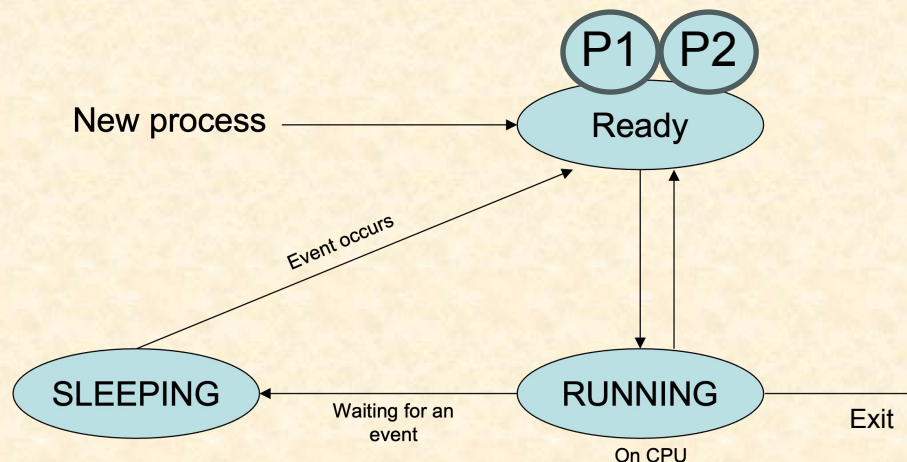
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$ ./P1 & ./P2
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

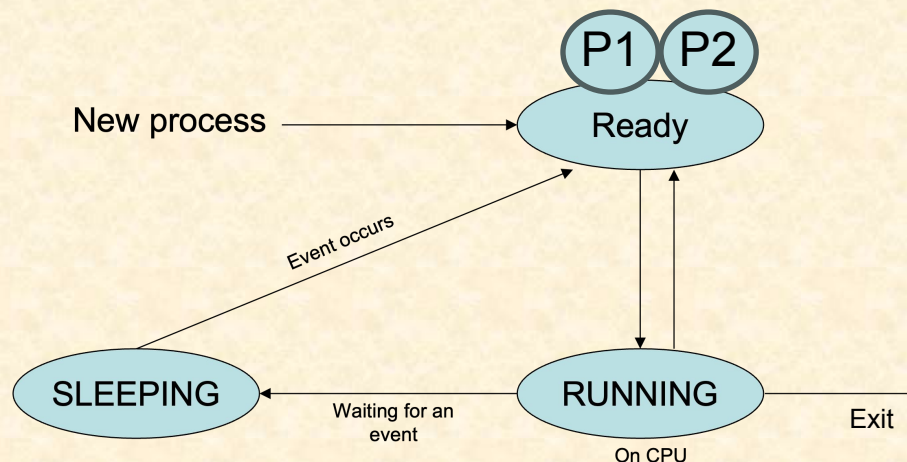
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

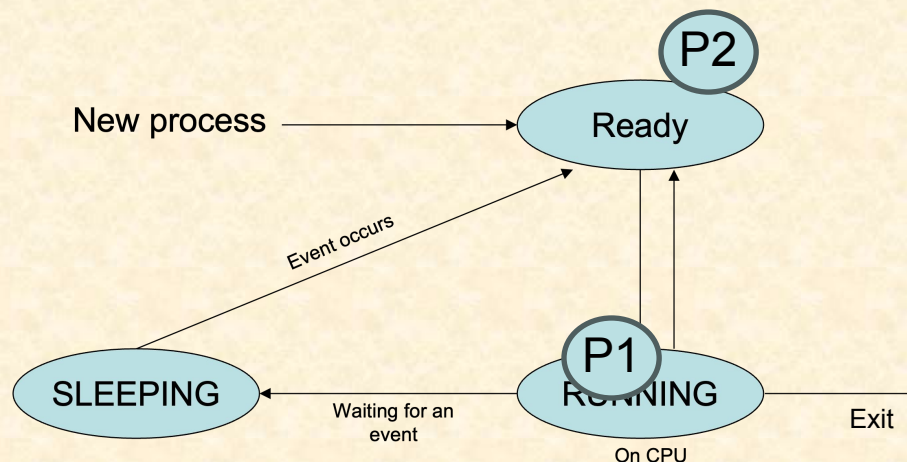
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

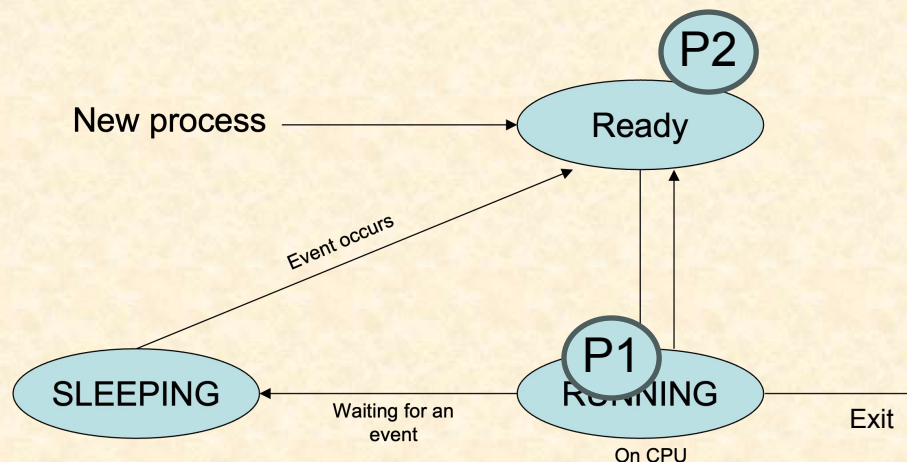
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 seconds
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

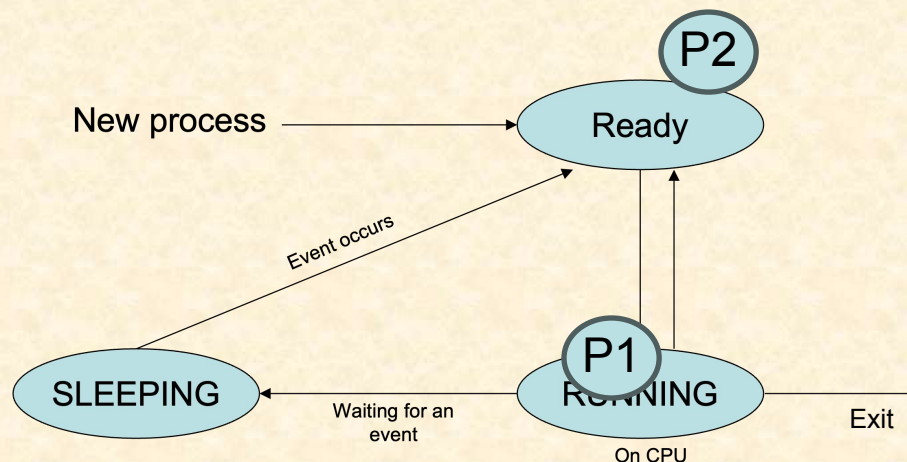
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 seconds
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

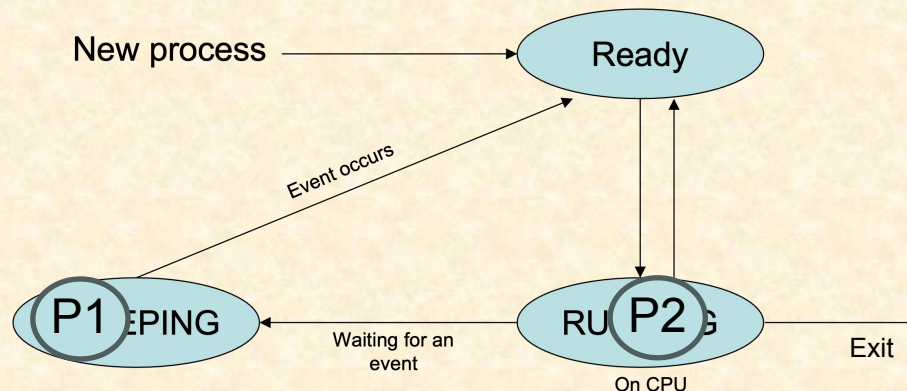
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

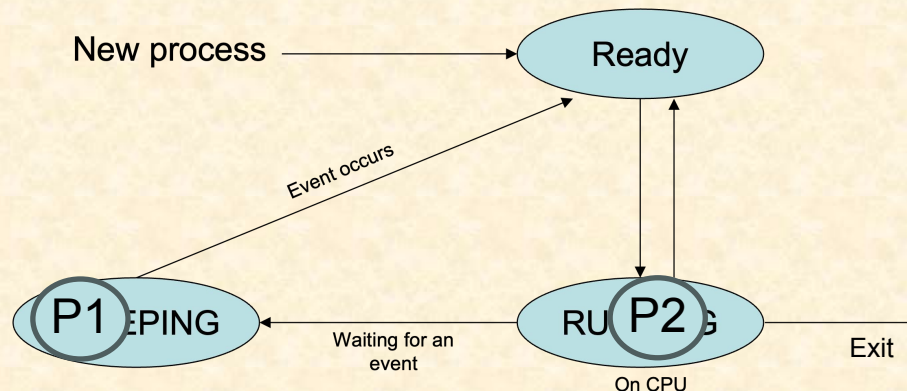
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

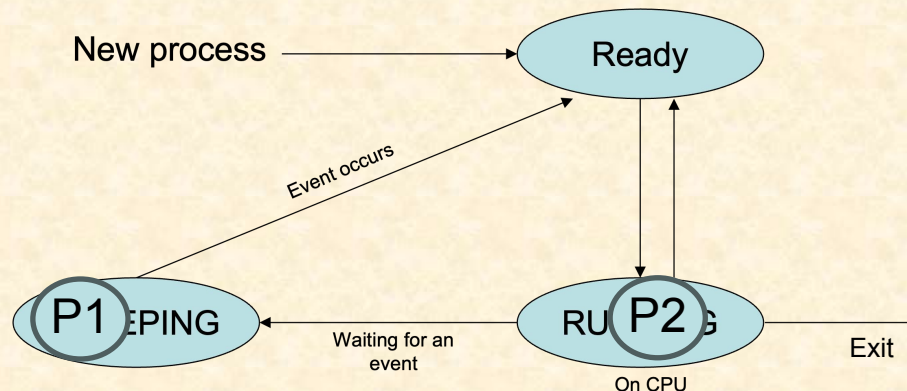
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



Who comes out of sleeping state first, P1 & P2?

State transition traces

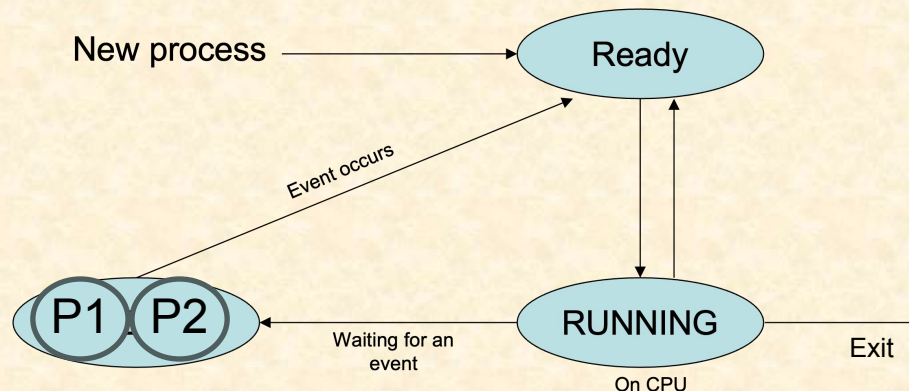
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

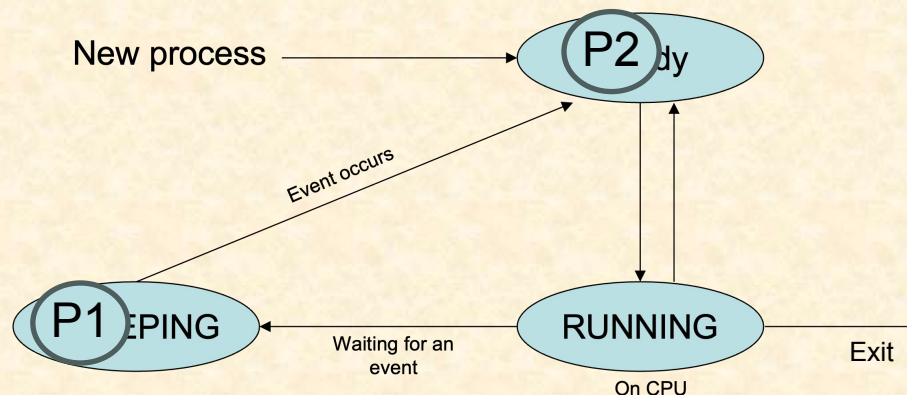
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
P2: 0
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

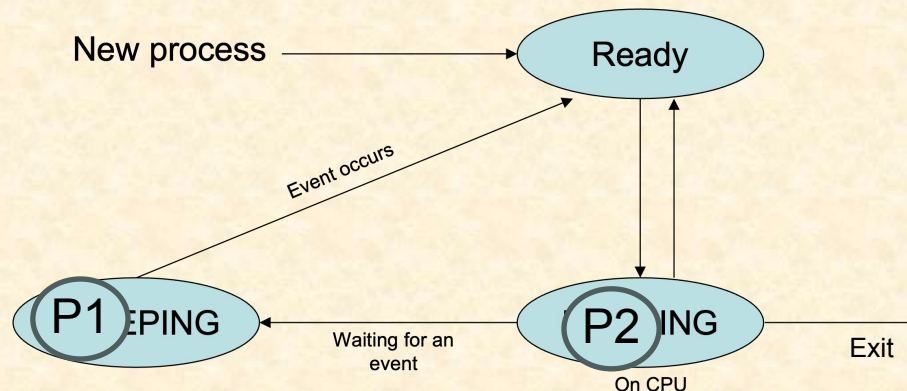
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
P2: 0
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

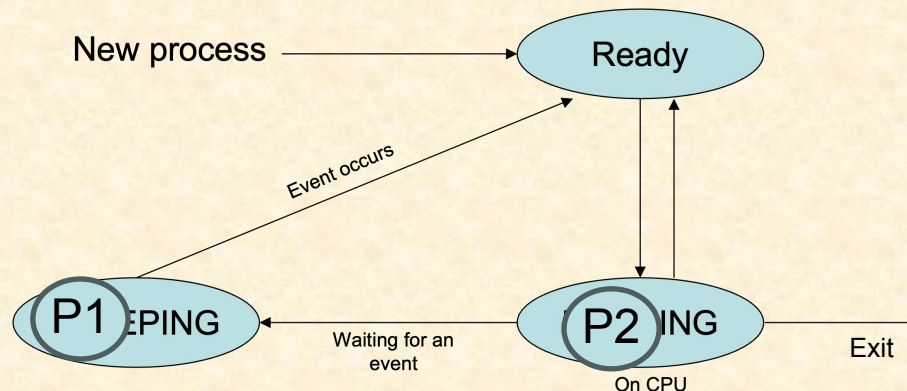
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
P2: 0
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

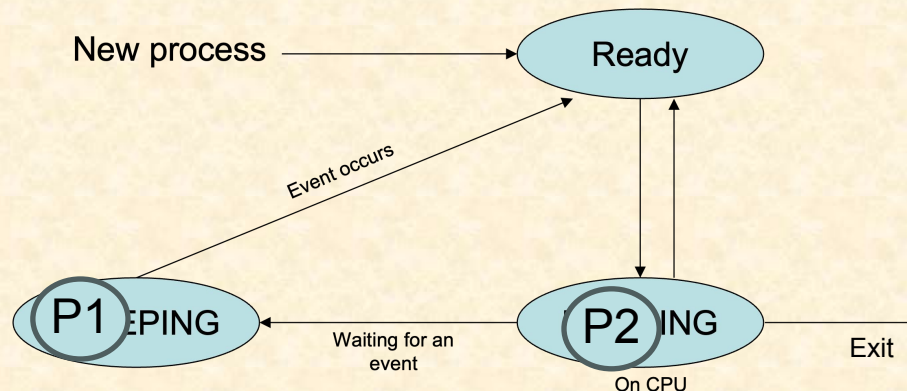
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
P2: 0
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

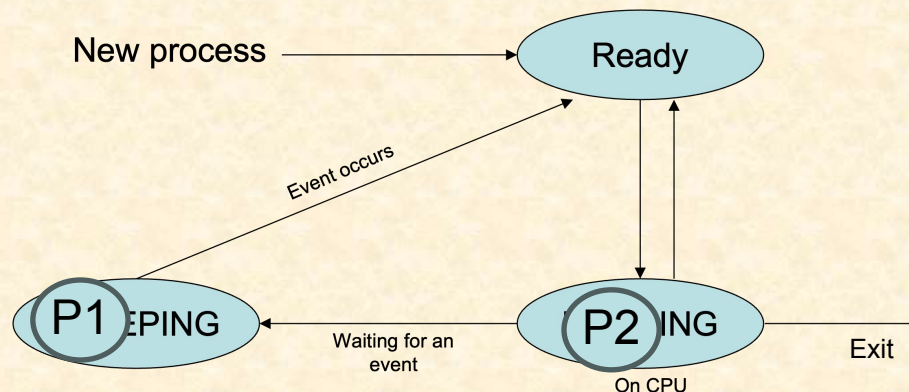
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
P2: 0
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



Who comes out of sleeping state first, P1 & P2?

State transition traces

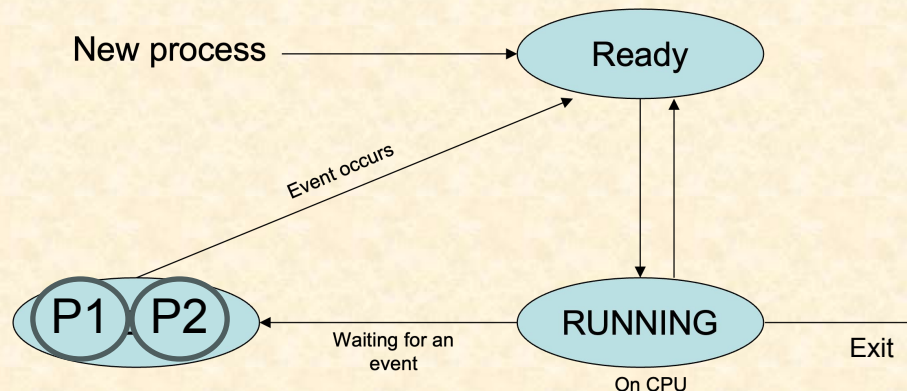
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
P2: 0
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

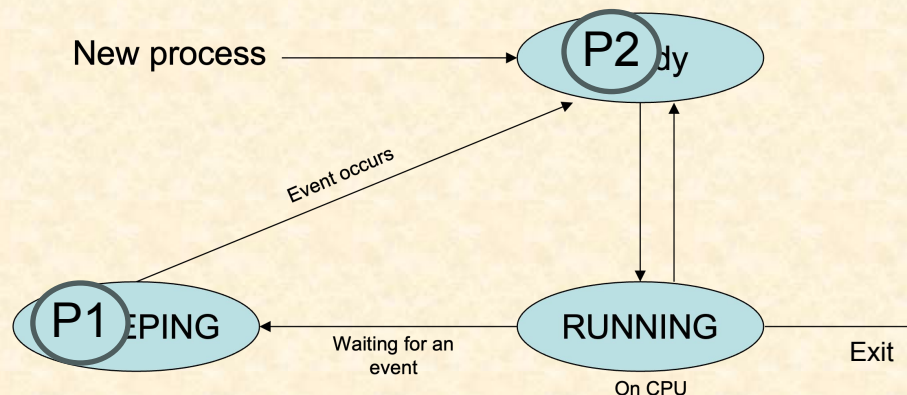
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
P2: 0
P2: 1
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

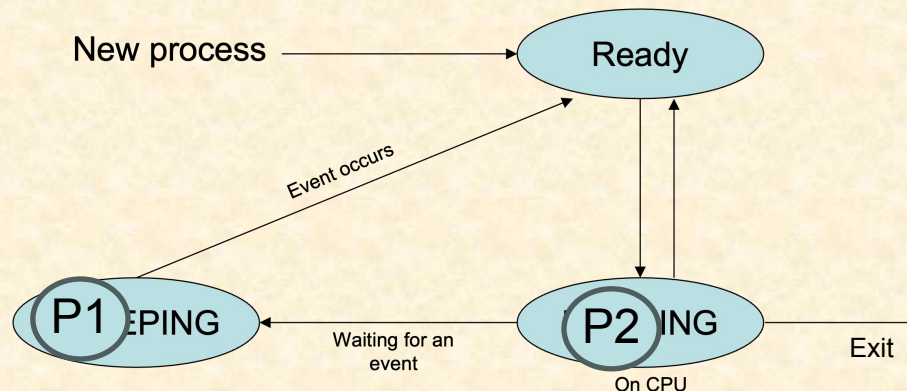
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
P2: 0
P2: 1
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

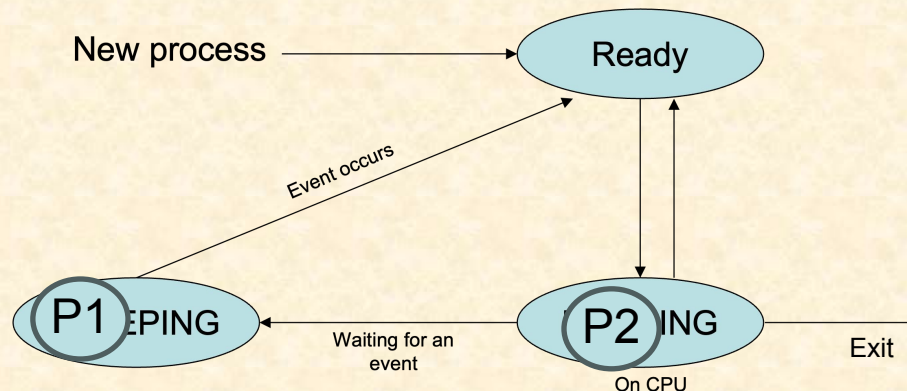
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
P2: 0
P2: 1
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

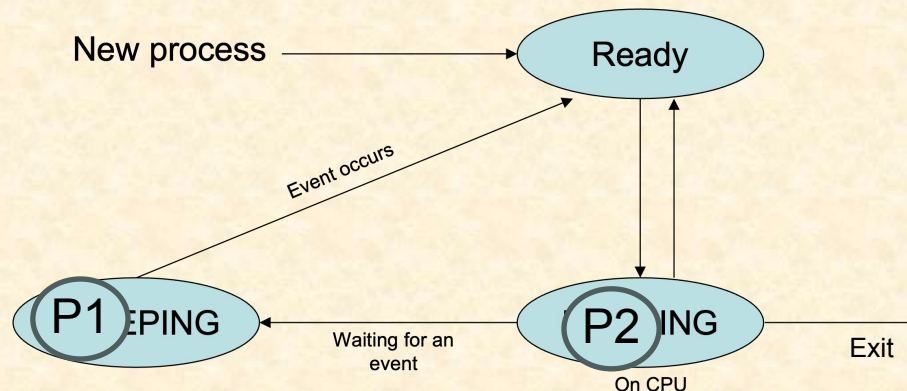
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
P2: 0
P2: 1
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

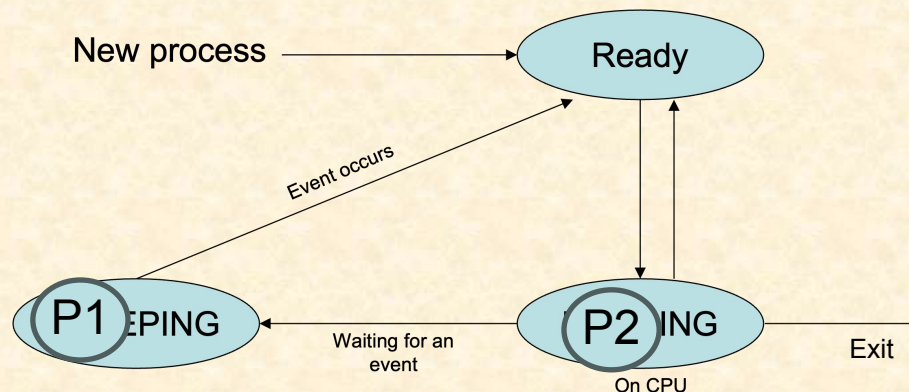
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
P2: 0
P2: 1
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



Who comes out of sleeping state first, P1 & P2?

State transition traces

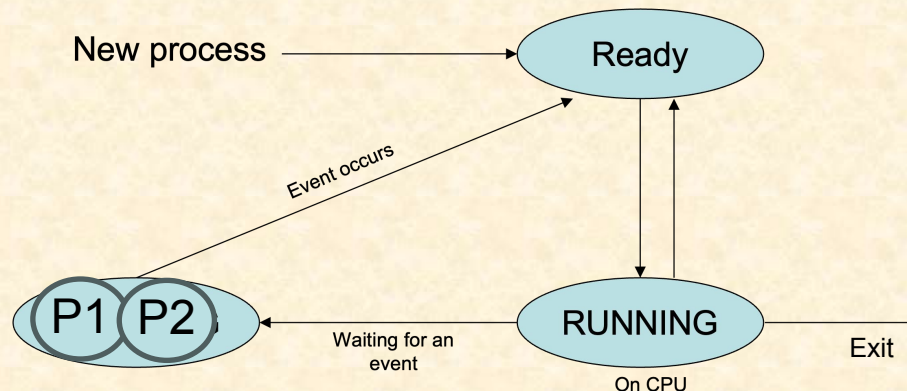
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
P2: 0
P2: 1
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

P2.c



State transition traces

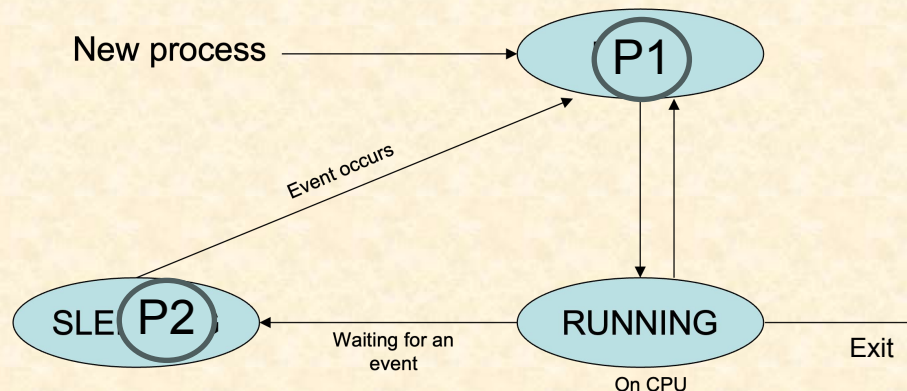
```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(2); //wait for 2 sec
    printf("P1:%d",i);
    i=i+1;
  }
}
```

P1.c

```
$ cc P1.c -o P1
$cc P2.c -o P2
$./P1 & ./P2
P2: 0
P2: 1
P1: 0
```

```
#include <stdio.h>
#include<unistd.h>
int main()
{ int i=0;
  while(i<10){
    sleep(1);
    printf("P2:%d",i);
    i=i+1;
  }
}
```

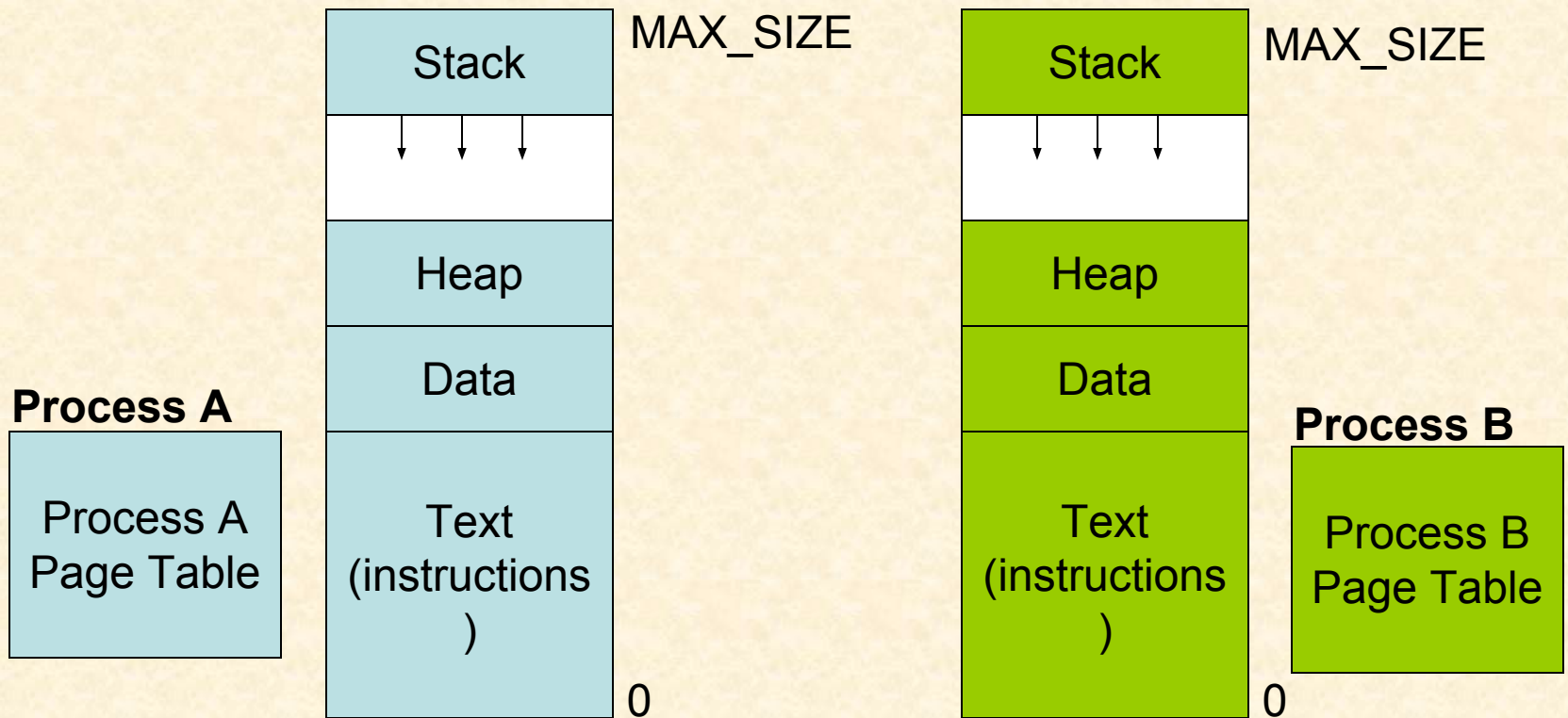
P2.c



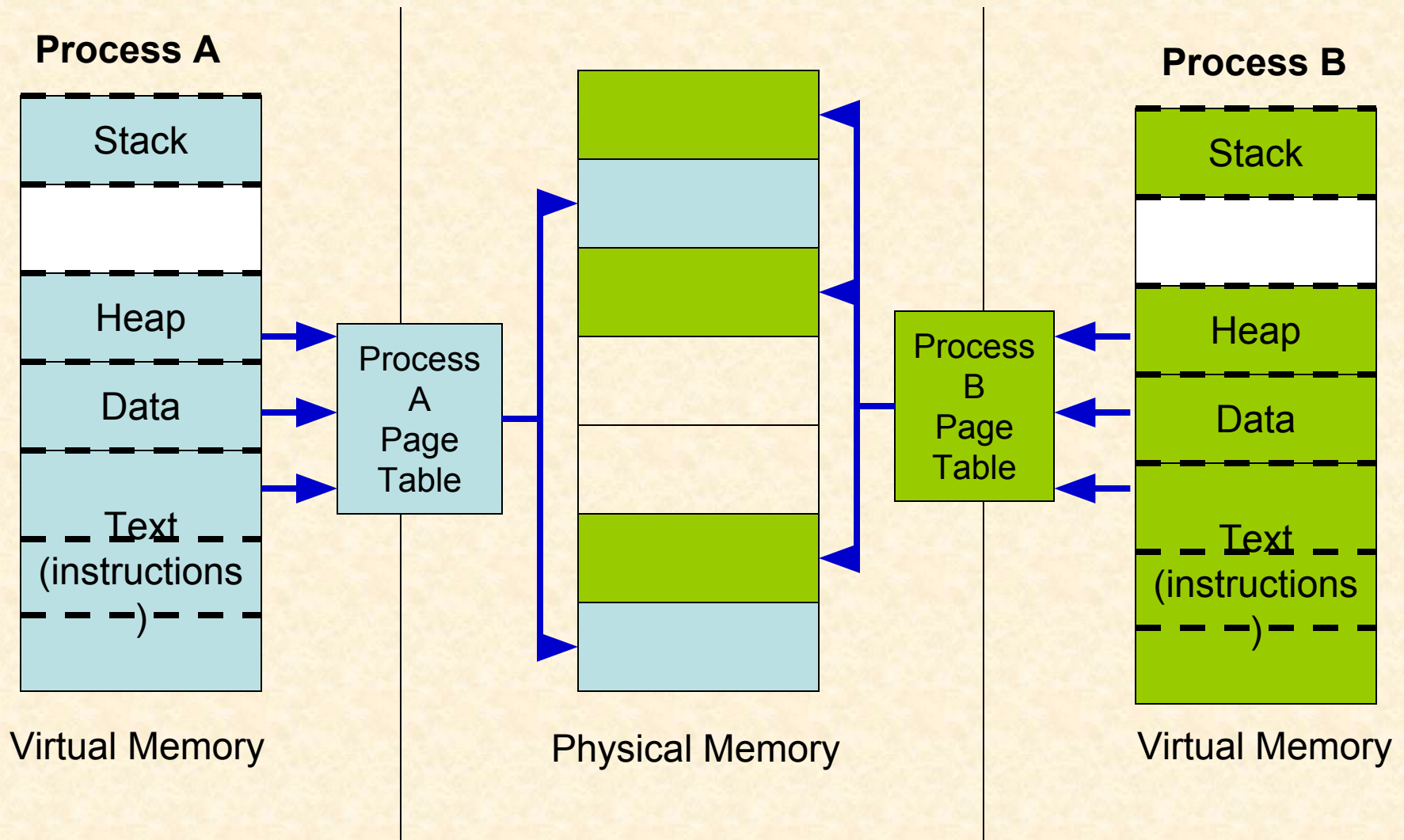
The transitions will continue until both processes complete

Process Address Space

- Each process has a different address space
- This is achieved by the use of virtual memory
- I.e. 0 to MAX_SIZE are virtual memory addresses



Virtual Address Mapping



ASIDE: KEY PROCESS TERMS

- The **process** is the major OS abstraction of a running program. At any point in time, the process can be described by its state: the contents of memory in its **address space**, the contents of CPU registers (including the **program counter** and **stack pointer**, among others), and information about I/O (such as open files which can be read or written).
- Processes exist in one of many different **process states**, including running, ready to run, and blocked. Different events (e.g., getting scheduled or descheduled, or waiting for an I/O to complete) transition a process from one of these states to the other.
- A **process list** contains information about all processes in the system. Each entry is found in what is sometimes called a **process control block (PCB)**, which is really just a structure that contains information about a specific process.

Next class System Calls: ch5 Remzi